



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3133 HIGH PERFORMANCE DATA PROCESSING

SECTION 01

PROJECT 2:

Real Time Sentiment Analysis using Apache Spark and Kafka

LECTURER: DR. MOHD SHAHIZAN BIN OTHMAN

Group Name: Fast&Furious

Student Name	Matric No.
GOE JIE YING	A23CS0224
NAWWARAH AUNI BINTI NAZRUDIN	A23CS0143
YASMIN BATRISYIA BINTI ZAHIRUDDIN	A23CS0201

Submission Date: 26 June 2026

Table of Contents

1.0 Introduction	4
1.1 Project Background	4
1.2 Problem statement	5
1.3 Objectives	5
1.4 Project Scope	6
2.0 Data Acquisition and Preprocessing	7
2.1 Data Source Description	7
2.2 Data Collection Method	8
2.3 Data Cleaning Steps	9
2.4 Final Dataset Description	11
3.0 Sentiment Model Development	13
3.1 Feature Engineering	13
3.1.1 TF-IDF Vectorisation	13
3.1.2 Label Encoding	14
3.2 Models Used	14
3.2.1 Multinomial Naive Bayes	14
3.2.2 Logistic Regression	14
3.2.3 XLM-RoBERTa	15
3.3 Training Process	15
3.3.1 Naive Bayes Training	15
3.3.2 Logistic Regression Training	15
3.3.3 XLM-RoBERTa Fine-Tuning	16
3.4 Evaluation Metrics	17
3.5 Model Comparison Table	17
3.6 Best Model Selection	19
4.0 Apache System Architecture	20
4.1 System Overview Diagram	20
4.2 Kafka Pipeline	20
4.2.1 Multi Listener Broker Configuration	20
4.2.2 Message Ingestion Mechanics	21
4.3 Spark Streaming	21
4.3.1 Ingestion and Parsing Strategy	21
4.3.2 Hybrid Micro-Batching with foreachBatch	22
4.4 Storage Layer	22
4.5 Dashboard Layer	22
4.5.1 Infrastructure Integration	22
4.5.2 Analytics Schema	23

5.0 Analysis and Results	24
5.1 Sentiment Analysis Results	24
5.2 Streaming vs Batch Analysis	25
5.3 Key Insights	26
6.0 Optimization and Comparison	30
6.1 Model Optimization	30
6.1.1 Naive Bayes	30
6.1.2 Logistic Regression	30
6.1.3 XLM-RoBERTa	31
6.2 Model Performance Comparison (Machine Learning vs Transformer)	32
6.3 Pipeline Optimization	33
6.4 Batch vs Streaming Performance Comparison	34
7.0 Conclusion and Future Work	36
7.1 Conclusion	36
7.2 Future work	37
8.0 References	38
9.0 Appendices	39

1.0 Introduction

This section provides a brief overview of the project background and objectives.

1.1 Project Background

In today's digital era, online platforms have become an important medium for people to express their opinions regarding social issues, products, services and government policies. These opinions are commonly shared through social media platforms via different methods, such as video-sharing websites, discussion forums and reels. The large volume of user-generated content provides insight into public sentiment and perception.

Sentiment analysis is the process of analyzing digital text using Artificial Intelligence (AI) algorithms and Natural Language Processing (NLP) to classify opinions expressed in the text into categories, for example, positive, negative or neutral. Organizations and governments increasingly rely on sentiment analysis to monitor public reactions, especially when they launch new policies or products to support data-driven decision making or make any improvement.

Traditionally, sentiment analysis relies on batch processing, which may not be suitable to handle continuously generated data streams, such as Youtube comments in the project. As the volume and velocity of online data continues to increase, the need for a scalable real-time processing system that is capable of analyzing sentiment efficiently is also increasing. Technologies such as Apache Kafka and Apache Spark become a powerful solution to process this kind of large-scale streaming data by enabling real-time data ingestion and presenting high-throughput analytics.

This project focuses on developing a real-time sentiment analysis pipeline using Apache Kafka and Apache Spark. In the initial phase, the system collects YouTube comments related to Malaysia's fuel subsidy policies through web scraping techniques. The collected comments are preprocessed using NLP techniques before being classified using Machine Learning and Transformer-based models. Consequently, the sentiment analysis pipeline is integrated with Apache Kafka, Apache Spark Structured Streaming, Elasticsearch and Kibana to support real-time sentiment monitoring and visualization.

1.2 Problem statement

With the launch of BUDI95, fuel subsidy policies continue to be a sensitive and popular topic among Malaysians. On social media like YouTube, a lot of public comments are constantly produced, making manual analysis inefficient and unfeasible.

Although sentiment analysis can automatically classify opinions, many existing systems focus only on model development and ignore real-time processing requirements. In addition, different models may produce different performance results depending on the data structure and contents, as well as the language complexity in the datasets.

Therefore, an integrated system capable of collecting, processing, classifying and visualizing sentiment data in real time is needed to handle continuous data streams while providing meaningful insights into public opinion.

1.3 Objectives

The objectives of this project are:

1. To collect Youtube comments related to Malaysia's fuel subsidy policies using web scraping techniques.
2. To preprocess and clean textual data collected using Natural Language Processing techniques, including tokenization and stop word removal.
3. To develop and evaluate sentiment classification models using machine learning and transformer-based approaches.
4. To compare the performance of selected models using metrics, like Accuracy, Precision, Recall and F1-Score.
5. To implement a real-time sentiment analysis pipeline using Apache Kafka and Apache Spark Structured Streaming.
6. To store sentiment classification results in Elasticsearch and visualize the results in Kibana dashboards.
7. To compare the pipeline performance of batch processing and streaming processing.

1.4 Project Scope

This project focuses on sentiment analysis of YouTube comments related to Malaysian fuel subsidy policies, particularly discussions regarding BUDI95. The comments are collected from 16 YouTube Videos using web scraping techniques, which includes both English and Malay comments, as well as mixed-language (Manglish) expressions. The comments are then processed using Natural Language Processing techniques, including text cleaning, Malay slang normalization, tokenization and stop-word removal.

Three classification models, Naive Bayes, Logistic Regression and XLM-RoBERTa, are developed and evaluated to classify comments into positive, negative and neutral sentiment categories. Furthermore, the project integrates Apache Kafka and Apache Spark Structured Streaming to support real-time analysis. The classified results are stored in Elasticsearch and visualized through Kibana dashboards to provide sentiment distribution and trend analysis.

2.0 Data Acquisition and Preprocessing

This section describes the data acquisition and preprocessing steps carried out to prepare the dataset for sentiment analysis. The process covers how YouTube comments were collected from publicly available videos related to Malaysia's fuel subsidy policy, followed by the cleaning and NLP preprocessing steps applied to the raw text before sentiment labelling and model training.

2.1 Data Source Description

The data used in this project consists of YouTube comments scraped from 16 videos related to Malaysia's fuel subsidy policies. The selected videos include news reports, government announcements, interviews, and public discussions uploaded by various Malaysian news channels and content creators.

Table 2.1 shows the list of videos scraped along with the number of comments collected from each video.

Table 2.1 YouTube Videos Used as Data Sources

No.	Video Title	Video URL	Comments Scraped
1	Diesel prices jump over 50% in Malaysia as blanket subsidies end	https://www.youtube.com/watch?v=LmfFs0KWJjA	262
2	The Malaysian Petrol Subsidy Might Be Removed. Here's Why	https://www.youtube.com/watch?v=ul7ljc7i-Mw	354
3	Ramai tak dapat bayangkan 300L byk mana? Boleh pegi 300KM je	https://www.youtube.com/watch?v=wpLz772Hq8A	622
4	Malaysia PM Anwar says govt will press ahead with petrol subsidy	https://www.youtube.com/watch?v=lSmNyl6ijTg	52
5	Malaysia di Ujung Krisis! Subsidi BBM Terancam Dipangkas	https://www.youtube.com/watch?v=rwlnFlFQ5y8	647
6	No plans to cut off T20 from Budi95 for now, says Transport Minister	https://www.youtube.com/watch?v=fWwEGDTr5x0	35
7	'T20' fuel subsidy cut: High	https://www.youtube.com/	210

	income does not mean rich, DAP	watch?v=4DAdEJuMGe0	
8	Malaysians banned from Budi95 for filling up container in car	https://www.youtube.com/watch?v=bMZdzKk7IyQ	159
9	Malaysia to maintain petrol subsidies; PM Anwar says supply sufficient	https://www.youtube.com/watch?v=eS2jmpIGDSE	62
10	Malaysia's Subsidies: Saving or Sinking the Economy?	https://www.youtube.com/watch?v=_Io9P79QA-k	272
11	Govt to finalise proposed review of fuel subsidies for the rakyat	https://www.youtube.com/watch?v=Hm3_evsVyCw	12
12	RON95 subsidy based on vehicle registration, not citizenship	https://www.youtube.com/watch?v=AzfWTbTIRSU	79
13	Malaysian government launches its most comprehensive fuel distribution system	https://www.youtube.com/watch?v=enRICXJam70	219
14	A targeted fuel subsidy will worsen inflation, says Najib	https://www.youtube.com/watch?v=KWL-JkFN8UY	27
15	[TERKINI] MOF sekat MyKad individu salah guna subsidi BUDI95	https://www.youtube.com/watch?v=mVmYibok-7A	335
16	Malaysia bracing for backlash over fuel subsidy revamp	https://www.youtube.com/watch?v=xdBzP3lgEnQ	101
Total Comments Scraped			3,448

2.2 Data Collection Method

The data was collected using a Python-based web scraping approach run on Google Colab. The main tool used was yt-dlp, an open-source command-line tool that can extract video metadata and comments from YouTube (yt-dlp Contributors, 2024). It was chosen because it supports large-scale comment extraction. The libraries used in this phase include yt-dlp for scraping, pandas for data handling, datetime for timestamp conversion and re (regular expressions) for URL parsing.

The scraping process followed these steps:

1. **URL preparation:** A list of 16 YouTube video URLs related to Malaysian fuel subsidy topics was prepared. A deduplication function was written using regular expressions to extract the video ID from each URL and remove any duplicate videos before scraping.
2. **Comment extraction:** For each unique video URL, yt-dlp was called with the getcomments option enabled. The extractor argument max_comments was set to 99,999 to collect all available comments.
3. **Metadata capture:** 10 fields were extracted for each comment, which are comment_id, parent_id, comment_type (top-level or reply), author, text, like_count, reply_count, published_at, video_id, and video_title.
4. **Reply count calculation:** Reply counts were calculated after scraping by grouping all replies by their parent_id and mapping the counts back to the top-level comments.
5. **Deduplication:** After combining all comments into a single DataFrame, duplicate entries based on comment_id were removed to ensure each comment appeared only once in the final dataset.
6. **Saving output:** The final raw dataset was saved as youtube_comments_raw.csv. A total of 3,448 unique comments were collected across 16 videos, with 1,778 top-level comments and 1,670 replies.

2.3 Data Cleaning Steps

The raw dataset was then processed through a text preprocessing pipeline implemented in a separate Google Colab notebook. The goal of this phase was to clean the raw comment text and prepare it for sentiment labelling and model training. The model used for labelling was XLM-RoBERTa, a multilingual transformer model pre-trained on over 100 languages and fine-tuned on social media text for three-class sentiment classification, which are negative, neutral, and positive (Barbieri et al., 2022).

The libraries used in this phase include transformers and torch for loading and running the XLM-RoBERTa model, sentencepiece for subword tokenisation, nltk for tokenisation, stop word removal, stemming, and lemmatisation, langdetect for language detection, pandas for data manipulation, matplotlib for visualisation, and re and os for text cleaning and file management.

The preprocessing pipeline followed these steps in order:

1. **Language Detection:** Each comment was analysed using the langdetect library to identify its primary language. Comments were categorised into four groups: English, Malay, Mixed, or Other. A set of common Malay keywords was also used as a secondary check to improve detection accuracy, since langdetect sometimes misclassifies short Malay or Manglish texts. This step added a language column to the dataset.
2. **Basic Text Cleaning:** A cleaning function was applied to remove noise from the raw comment text. This included:
 - Removing URLs (http and www links)
 - Removing email addresses
 - Removing numbers
 - Removing special characters and punctuation (replaced with spaces)
 - Converting all text to lowercase
 - Removing extra whitespace
3. **Malay Slang Normalisation:** A custom dictionary of 26 Malay slang rules was applied using regular expressions to expand common abbreviations and informal terms used in Malaysian online conversations. For example, tkde was replaced with tidak ada, xnak with tidak mahu, and filler words like haha, lol, and wkwk were removed. This step helped standardise informal Malay text before tokenisation.
4. **Tokenization:** The cleaned text was tokenised using NLTK's RegexpTokenizer, which extracts words with at least 3 characters. This automatically filters out very short tokens such as single letters or two-character words that carry little meaning.
5. **Stop Word Removal:** A combined stop word list was created by merging NLTK's English stop words with a custom list of 66 Malay stop words. Any token found in this combined list of 305 words was removed from the tokenised output.
6. **Stemming and Lemmatisation:** For English comments, both stemming and lemmatisation were applied to reduce words to their base forms. This step was skipped for Malay and Mixed comments because Malay morphology is different and English stemming rules do not apply correctly to Malay text.
7. **XLM-RoBERTa Sentiment Labelling:** Sentiment labels were assigned to each comment using the pre-trained XLM-RoBERTa model. This model outputs probabilities

for three classes: negative, neutral, and positive. A Neutral-Shrinking Threshold of 0.40 was applied. A comment was only labelled as neutral if its neutral probability was at least 0.40 and higher than both the negative and positive probabilities. Otherwise, the comment was forced into either positive or negative based on whichever had the higher probability. This threshold was used to reduce over-prediction of neutral labels, which is a common issue with social media text.

8. **Filtering Short Comments:** After cleaning, comments where the `cleaned_text` was shorter than 6 characters were removed, as they contained no meaningful content for analysis, such as emoji-only or single-word comments that became empty after stop word removal.

2.4 Final Dataset Description

After preprocessing, the final dataset contained 3,349 comments, down from the original 3,448 due to the removal of 99 comments that were too short after cleaning. Table 2.2 describes the columns included in the final dataset file.

Table 2.2 Final Dataset Columns

No.	Column	Description
1	<code>comment_id</code>	Unique YouTube comment identifier
2	<code>comment_type</code>	Whether the comment is a top-level comment or a reply
3	<code>author</code>	YouTube username of the commenter
4	<code>language</code>	Detected language (English, Malay, Mixed, or Other)
5	<code>cleaned_text</code>	Preprocessed text after data cleaning steps
6	<code>sentiment</code>	Predicted sentiment label (positive, negative, or neutral)
7	<code>polarity</code>	Continuous sentiment score, ranging from -1 to 1
8	<code>prob_neg</code>	XLM-RoBERTa probability score for negative class
9	<code>prob_neu</code>	XLM-RoBERTa probability score for neutral class
10	<code>prob_pos</code>	XLM-RoBERTa probability score for positive class

11	like_count	Number of likes on the comment
12	reply_count	Number of replies to the comment
13	published_at	Date and time the comment was posted
14	video_id	YouTube video identifier
15	video_title	Title of the YouTube video the comment belongs to

3.0 Sentiment Model Development

This section describes the development of the sentiment classification models used in the real-time sentiment analysis system. Three classification models (Multinomial Naive Bayes, Logistic Regression, and XLM-RoBERTa) were implemented and evaluated.

3.1 Feature Engineering

Before model training, the raw text data must be converted into a numerical format for the machine learning algorithms to process. The dataset consists of 334 pre-cleaned YouTube comments. The primary feature engineering task in this phase is vectorization since the text has undergone a cleaning process in the previous phase.

3.1.1 TF-IDF Vectorisation

Term Frequency–Inverse Document Frequency (TF-IDF) was applied to transform the cleaned text into a weighted numerical matrix. TF-IDF assigns higher weights to words that are important in a document but less common across the entire dataset, while reducing the influence of commonly used but less meaningful words. This is useful as common words such as “the” or “and” do not bring any meaningful sentiment analysis. Table 3.1.1 describes the parameters used to configure TF-IDF vectoriser.

Table 3.1.1 Parameters Used to Configure TF-IDF

Parameter	Value	Justification
max_features	15,000	Limits vocabulary size for efficiency and reduces noise
ngram_range	(1, 2)	Captures both single words and word pairs for better context
sublinear_tf	True	Applies logarithmic scaling to term frequency to reduce bias from repeated words
min_df	2	Removes rare words that appear in fewer than 2 documents

After vectorisation, the resulting TF-IDF matrix had a shape of $(3,349 \times 7,531)$, meaning 3,349 comments were represented using 7,531 features. This TF-IDF representation was used as input for the Naive Bayes and Logistic Regression models. However, XLM-RoBERTa does not require

TF-IDF because it uses its own internal subword tokenisation method (SentencePiece BPE) to directly process raw text.

3.1.2 Label Encoding

The comments were classified into three sentiment classes, negative, neutral and positive. Each class was encoded into integer labels, 0,1 and 2, respectively using LabelEncoder. Using this mapping, the models can process categorical targets numerically in a machine readable format. The dataset shows a class imbalance, with negative comments making up 61.5% of the data, neutral 26.6%, and positive only 11.9%. This imbalance was addressed differently across each model as described in Section 3.3.

3.2 Models Used

The sentiment classification models with different levels of complexity were selected, namely Naive Bayes, Logistic Regression and XLM-RoBERTa. This ensures a fair comparison between traditional machine learning models and modern transformer-based models for the BUDI95 YouTube comment sentiment classification task.

3.2.1 Multinomial Naive Bayes

Multinomial Naive Bayes is a machine learning algorithm commonly used for text classification tasks such as spam detection and sentiment analysis. The model predicts sentiment based on the frequency of words that occurs in a comment. It is simple and computationally efficient, especially on text data represented using TF-IDF features. In this project, Multinomial Naive Bayes was selected as a baseline model to provide a benchmark for comparison.

3.2.2 Logistic Regression

Logistic Regression is a supervised machine learning algorithm that predicts the probability of a comment belonging to a particular sentiment class. Unlike Naive Bayes, it considers all features contributions when predicting to achieve higher classification accuracy. In this project, Logistic Regression was used as an improved machine learning model to determine whether a more advanced classifier might more accurately predict sentiment trends in YouTube comments.

3.2.3 XLM-RoBERTa

XLM-RoBERTa is a transformer-based language model developed for multilingual text understanding. The model has been pre-trained on over 100 languages, including Malay and English which are mainly used in our case to understand Manglish comments. In contrast to classical machine learning models that mostly depend on word frequency, XLM-RoBERTa can capture the contextual meaning of words inside a phrase. In this project, the model was fine-tuned using the collected YouTube comments to strengthen its ability to classify sentiments related to the BUDI95 fuel subsidy policy.

3.3 Training Process

All models were trained on the same stratified dataset split to ensure fair comparison. Table 3.3 shows the distribution of training, validation and test sets using stratified sampling to preserve the original class ratios in each split.

Table 3.3 Distribution of Training, Validation and Test Sets

Split	Ratios	Samples (Total)
Training	70%	2344
Validation	10%	335
Test	20%	670

3.3.1 Naive Bayes Training

Naive Bayes was fitted directly on the TF-IDF training matrix in a single pass. Beyond the smoothing value ($\alpha = 0.1$), no hyperparameter adjustments were necessary. The model's closed-form estimation of class priors and conditional probabilities made training practically fast.

3.3.2 Logistic Regression Training

Logistic regression was trained using the saga solver with multinomial cross-entropy loss and L2 regularisation. Inverse-frequency weights were automatically calculated using the `class_weight='balanced'` parameter. A maximum of 1000 iterations was allowed for convergence. To speed up training, the model was parallelized over all CPU cores (`n_jobs = -1`).

3.3.3 XLM-RoBERTa Fine-Tuning

XLM-RoBERTa was fine-tuned using the HuggingFace Transformers Trainer API. To apply weighted cross-entropy loss, a customer Weighted Trainer subclass was implemented. Table 3.3.3.1 shows the class weights computed.

Table 3.3.3.1 Class Weighted Computed in XLM-RoBERTa Weighted Trainer

Split	Ratios	Samples (Total)
Training	70%	2344
Validation	10%	335
Test	205	670

The fine-tuning hyperparameters used in XLM-RoBERTa were demonstrated in Table 3.3.3.2. The selected hyperparameters were chosen to balance training efficiency, model convergence and generalization performance on the sentiment classification task.

Table 3.5.1 Performance of Naive Bayes, Logistic Regression and XLM-RoBERTa

Hyperparameter	Value	Rationale
Learning Rate	2e-5	Standard for transformer fine-tuning
Epochs	Up to 8	Prevents overfit on small dataset
Batch size	16	Balances GPU memory and gradient stability
Max sequence length	64 tokens	Sufficient for short social media comments
Weight decay	0.01	Regularisation to prevent overfitting
Warmup steps	50	Gradual LR increase at start of training
Early stopping patience	3 epochs	Stops if validation F1 does not improve
Best model criterion	Validation F1-score	Most robust metric for imbalanced classes
Mixed precision (fp16)	Enabled (CUDA)	Halves memory usage and speeds up training

The training was run for all 8 epochs and stopped early when the validation performance failed to improve for 3 epochs. The validation F1-score increased steadily from 0.477 in epoch 1 to 0.703 in epoch 7, with no further improvement observed in epoch 8, indicating model

convergence. Consequently, the best-performing checkpoint from epoch 7 was automatically loaded for final evaluation.

3.4 Evaluation Metrics

All three models were evaluated on the same test set of 670 samples using four classification metrics, including accuracy, precision, recall and weighted F1-score. Among them, weighted F1-score was selected as the main model selection criterion because of the major class imbalance in the dataset, where negative comments are dominant at 61.5% while positive comments only have 11.9%. In this situation, if based on raw accuracy alone would be deceptive as a model that constantly makes negative predictions would already get above 60% accuracy without learning any meaningful distinction between classes.

To ensure that each class's contribution is equal to the support in the test set, weighted averaging was applied for all measures. This reflects the realistic distribution of public sentiment in BUDI95-related YouTube comments and guarantees that minority positive class with 80 samples only is not overlooked.

3.5 Model Comparison Table

Table 3.5.1 summarizes the performance of all three models, including Naive Bayes, Logistic Regression and XLM-RoBERTa on the test set. All metrics are reported as weighted averages across classes. Table 3.5.2 to Table 3.5.4 shows the performance for the all selected model and it's categorized by class.

Table 3.5.1 Performance of Naive Bayes, Logistic Regression and XLM-RoBERTa

Model	Accuracy	Precision	Recall	F1-Score
Naive Bayes	0.6388	0.5996	0.6388	0.5987
Logistic Regression	0.6373	0.6350	0.6373	0.6357
XLM-RoBERTa	0.7403	0.7365	0.7403	0.7381

Table 3.5.2 Per-class performance for Naive Bayes

Class	Precision	Recall	F1-Score	Support
negative	0.68	0.86	0.76	412
neutral	0.53	0.38	0.44	178
positive	0.37	0.09	0.14	80

Table 3.5.3 Per-class performance for Logistic Regression

Class	Precision	Recall	F1-Score	Support
negative	0.75	0.75	0.75	412
neutral	0.49	0.53	0.51	178
positive	0.36	0.30	0.33	80

Table 3.5.4 Per-class performance for XLM-RoBERTa

Class	Precision	Recall	F1-Score	Support
negative	0.81	0.83	0.82	412
neutral	0.66	0.65	0.66	178
positive	0.53	0.47	0.50	80

Naive Bayes achieved a weighted F1-score of 0.5987, with relatively poor performance on the positive class with F1-score of 0.14, as its aggressive prediction on the dominant negative class. Logistic regression, which is slightly better than Naive Bayes, with a weighted F1-score of 0.6357 and better management of the minority classes due to its balance class weighting. XLM-RoBERTa significantly outperformed the other two classical models, with a weighted F1-score of 0.7381 and accuracy of 74.03%. Its strong results on the negative class (F1 = 0.82) and moderate results on neutral (F1 = 0.66) and positive (F1 = 0.50) proves its ability on using multilingual pretraining for Manglish text.

3.6 Best Model Selection

The best model was selected based on the weighted F1-Score and accuracy. Table 3.6 shows the weighted F1-Score and accuracy for the best model - **XLM-RoBERTa**.

Table 3.6 Weighted F1-Score and Accuracy for XLM-RoBERTa

Selected Model	XLM-RoBERTa
Weighted F1-Score	0.7381
Accuracy	74.03%

XLM-RoBERTa was selected as the final sentiment classification model due to its superior performance compared to Naive Bayes and Logistic Regression. XLM-RoBERTa achieved the highest weighted F1-score of 0.7381, outperforming Naive Bayes with 0.5987 and Logistic Regression with 0.6357. Since the dataset was imbalanced, with approximately 61.5% of negative comments and 11.9% of positive comments in the data, the weighted F1-score became a more reliable evaluation metric than accuracy as it does not bias towards the majority class.

Besides, XLM-RoBERTa is a multilingual transformer model pre-trained on more than 100 languages, including Malay in the YouTube comments data. This allows the model to understand and process multilingual, especially Manglish comments that commonly appeared in Malaysian online discussions. Also, fine-tuning the model on comments related to the BUDI95 fuel subsidy policy further improved its ability to capture domain-specific sentiment patterns.

During the manual probe testing process, the model demonstrated strong predictive capability by correctly classifying both English and Malay comments with confidence scores of up to 97.6%. Based on its overall performance and multilingual capabilities, XLM-RoBERTa was selected as the deployment model for the real-time sentiment analysis pipeline. The fine-tuned model, tokenizer and configuration files were saved and integrated into Apache Spark Structured Streaming workflow to enable real-time sentiment prediction on streaming comments received from Kafka before the results were stored in Elasticsearch for visualization in Kibana.

4.0 Apache System Architecture

The architecture of this project uses three major technologies in the Apache ecosystem, which are Apache Kafka, Apache Spark, and Elasticsearch. These components work along to produce a robust pipeline for streaming data, real-time processing and finally provide a dashboard. This section explains each component's roles and how they work together.

4.1 System Overview Diagram

The system consist of three major parts:

1. **Apache Kafka:** Acts as the high throughput, fault-tolerant message broker. It decouples the data source from processing engine, receiving incoming YouTube comments sequentially and hosting them in a distributed queue topic.
2. **Apache Spark:** Used Spark Structured Streaming to ingest micro-batches of data from Kafka in real time. It works by performing JSON schema parsing, deserialization, and coordinate inference through a deep learning sentiment model to enrich the raw stream.
3. **Elasticsearch:** Serves as the distributed search and analytics engine. It receives the enriched JSON documents (contains the original text, metadata, predicted sentiment labels, and confidence scores) via bulk indexing, making the data instantly queryable for downstream dashboards.

4.2 Kafka Pipeline

The ingestion layer used Apache Kafka to manage real-time streams asynchronously, ensuring high throughput, durability, and a decoupling of components.

4.2.1 Multi Listener Broker Configuration

The streaming infrastructure runs within a containerized environment deployed via Docker Compose. The Kafka broker image is linked with Apache Zookeeper for distributed synchronization, metadata tracking, and cluster state management. To enable cross network communication, the broker is configured with dual-listener architecture using security protocol map.

- Internal Container Network (PLAINTEXT://kafka:29092):

Handles internal inter-container data transfers and management operations within the isolated Docker bridge network.

- External (PLAINTEXT_HOST://localhost:9092): Expose the broker to application scripts executing directly on the host machine.

4.2.2 Message Ingestion Mechanics

The engine initializes a KafkaProducer configured with a linger_ms optimization metric set to 10. This is to ensure that messages are lightly batched before network dispatching rather than pushed instantly, reducing I/O operation.

- Topic Routing
Messages are written directly into the youtube_comments Kafka topic. The topic is configured with KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true", guaranteeing automated initialization upon producer execution.
- Simulated Stream Velocity
A configurable - - delay flag defaults to 0.2 seconds between entries to replicate live ingestion speeds, meanwhile - - limit flag enable subset boundaries to perform stable streaming benchmarks.

4.3 Spark Streaming

Apache Spark serves as the streaming engine, ingesting data via Spark Structured Streaming through a native spark-sql-kafka-0-10_2.12 connector package.

4.3.1 Ingestion and Parsing Strategy

The execution script builds a local SparkSession configured with a reduced partition allocation to optimize CPU usage on local machines. The processing sequence executes as follows:

1. Stream Loading: Streaming dataframe subscribe to the youtube-comments topic, pulling unread data from the latest available offsets.
2. Type Casting: The raw Kafka binary payload value attribute is cast to a string formatting.

3. Schema Extraction: The stringified JSON is passed into `from_json()`, applying a rigid `StructType` mapping to unpack the nested values into top-level columns

4.3.2 Hybrid Micro-Batching with `foreachBatch`

Spark is executed on the host system outside of Docker, while Kafka and Elasticsearch are containerized. This structure is chosen because the heavy deep learning dependency packages reside in the host machine's virtual environment (venv).

In `local[*]` mode, `foreachBatch` runs on the DRIVER. That means the model is loaded ONCE in this process and reused for every micro-batch, with no per-row reloading and no tricky serialisation of a PyTorch model out to executors. It is the most robust setup for a laptop and keeps throughput sane. Each micro-batch is small, so collecting it for the driver is fine.

4.4 Storage Layer

The storage architecture leverages Elasticsearch acting as a real-time, distributed document indexing engine.

Once sentiment inference is computed on the Spark driver, data is structured into Elasticsearch action documents. The downstream delivery uses the `elasticsearch.helpers.bulk` API. By compiling an entire micro-batch array of JSON dictionaries and executing a single HTTP request, the storage layer eliminates individual transaction overhead and maximizes i/o speeds.

4.5 Dashboard Layer

The last layer uses Kibana to read data directly from the underlying Elasticsearch cluster.

4.5.1 Infrastructure Integration

Kibana is linked directly to the Elasticsearch service as it is deployed as a dependent container service. They linked via internal networking over port 9200. The dashboard user interface is exposed to the host machine on port 5601. This layout provides immediate web browser accessibility at <http://localhost:5601>

4.5.2 Analytics Schema

Kibana connects directly to the underlying index to track and extract key data fields such as sentiment vs. true_sentiment where it captures and pairs predicted sentiment labels against ground truth labels. This enables real-time accuracy profiling, matrix visualization, and predictive reporting directly within the dashboard.

5.0 Analysis and Results

This section presents the analysis and results obtained from the sentiment classification models and the real-time streaming pipeline. It covers the sentiment distribution of the dataset, the performance results of the three classification models, a comparison between batch and streaming processing, and key insights drawn from the system.

5.1 Sentiment Analysis Results

The sentiment analysis results from the XLM-RoBERTa labelling step reveal patterns in how the Malaysian public responded to BUDI95-related content on YouTube.

Figure 5.1 shows that out of 3,349 comments, 2,060 (61.5%) were classified as negative, 890 (26.6%) as neutral, and 399 (11.9%) as positive. The strong dominance of negative sentiment suggests that public reaction to Malaysia's fuel subsidy changes was largely critical. Many commenters expressed concerns about rising living costs, the fairness of the subsidy system, and the impact on lower-income groups.

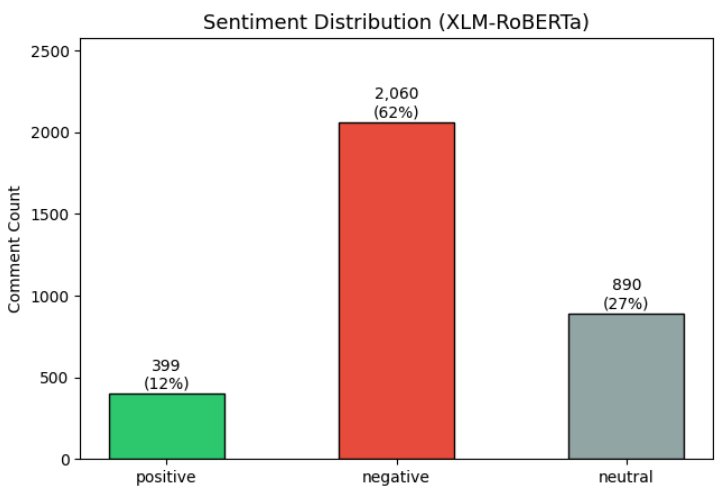


Figure 5.1 Sentiment Distribution

The dataset contained a near-equal split between English (47.1%) and Malay (46.9%) comments, with a small proportion of other languages (5.8%) and mixed-language comments (0.2%) as shown in Figure 5.2. This multilingual nature of the dataset reflects how Malaysians typically communicate online. They are switching between English and Malay depending on the context.

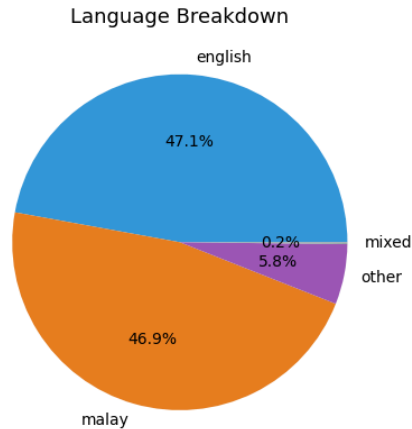


Figure 5.2 Language Breakdown of the Comment Section

5.2 Streaming vs Batch Analysis

The pipeline was evaluated under two processing modes using the same 3,349 BUDI95 YouTube comments and the same Logistic Regression model, ensuring a controlled, fair comparison.

In batch mode as shown in Figure 5.2.1, all 3,349 comments were loaded into memory and classified in a single vectorized pass using the trained model, with no inter-system processing time of 0.065 seconds and a throughput of 51506.2 records per second.

In streaming mode as shown in Figure 5.2.2, comments were ingested through Apache Kafka and processed by Apache Spark Structured Streaming in micro-batches. The first micro-batch of 200 records took 3.798 seconds, initially due to the one-time model loading cost. The second micro-batch of 3,149 records completed in 0.175 seconds at approximately 17,999 records per seconds. End to end, streaming processes all 3,349 comments in 3.973 seconds at approximately 843 records per seconds.

Even though the throughput differed, both modes produced identical classification accuracy of 81.19%, confirming that the streaming pipeline introduced no loss of correctness. The key distinction is purpose, batch mode maximizes throughput for complete historical datasets, while streaming enables real-time classification of data as it arrives. It's straight lying in latency rather than raw throughput.

```

=====
BATCH MODE - PERFORMANCE + RESOURCES
=====
Records processed : 3,349
Total time       : 0.065 s
Throughput      : 51506.2 records/sec
Accuracy        : 0.8119
CPU usage       : 86.9% (of one core)
Peak memory (RSS) : 144.0 MB
=====
Note: CPU% is relative to a single core, so values above 100%
mean multiple cores were used. Memory is this process only.

```

Figure 5.2.1 Batch mode

```

[sentiment_model] loaded 'sklearn' (classical) from saved_models
[batch 1] 200 records | 3.798s | 52.7 rec/s -> ES
[batch 2] 3149 records | 0.175s | 17998.9 rec/s -> ES

```

Figure 5.2.2 Stream mode

5.3 Key Insights

- Insight 1: Public sentiment toward BUDI95 is predominantly negative

As shown in Figure 5.3.1, the sentiment distribution pie chart. 56.11% of the 3,349 classified YouTube comments were negative, 30.31% neutral, and 13.59% positive. This visualization clearly illustrates that negative sentiment accounts for more than half of all public reactions, making it the dominant sentiment by a significant margin. Positive sentiment, at only 13.59% represents the smallest share, meaning that favourable responses to BUDI95 are relatively rare in the collected dataset.

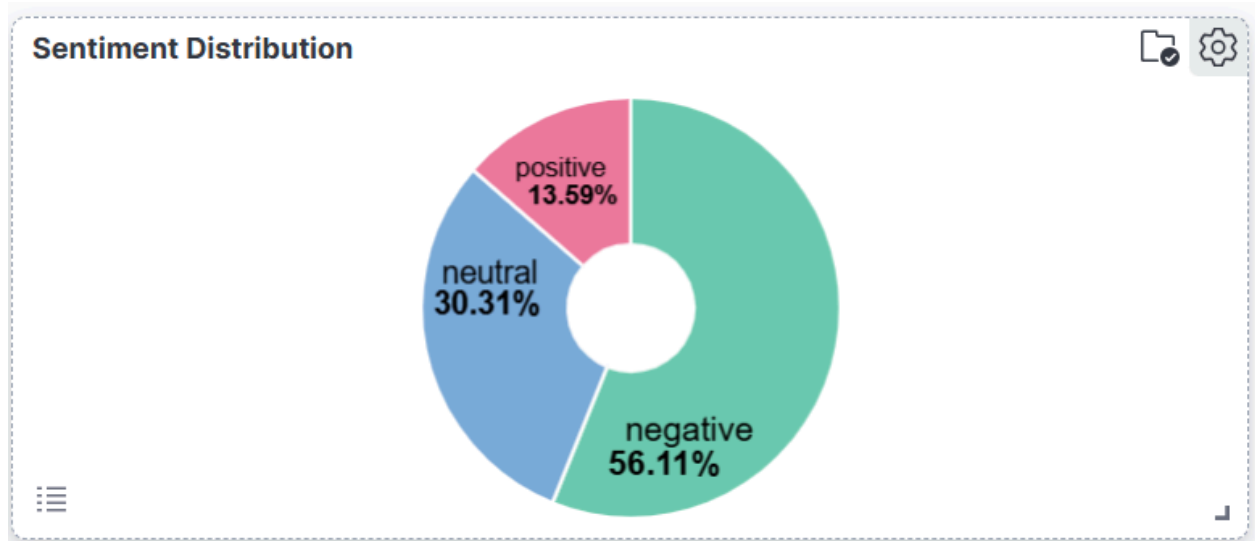


Figure 5.3.1 Pie Chart Insight 1

- Insight 2: The volume and scale of sentiment is confirmed by absolute counts

As shown in Figure 5.3.2, the sentiment metrics panel reinforces this finding with 1,879 negative comments, 1,015 neutral comments, and 455 positive comments out of 3,349 total. By having these figures, it confirms that the negative sentiment dominance is not a statistical volume of critical public opinion. The metrics panel provides the scale context that complements the proportional view of the pie chart.

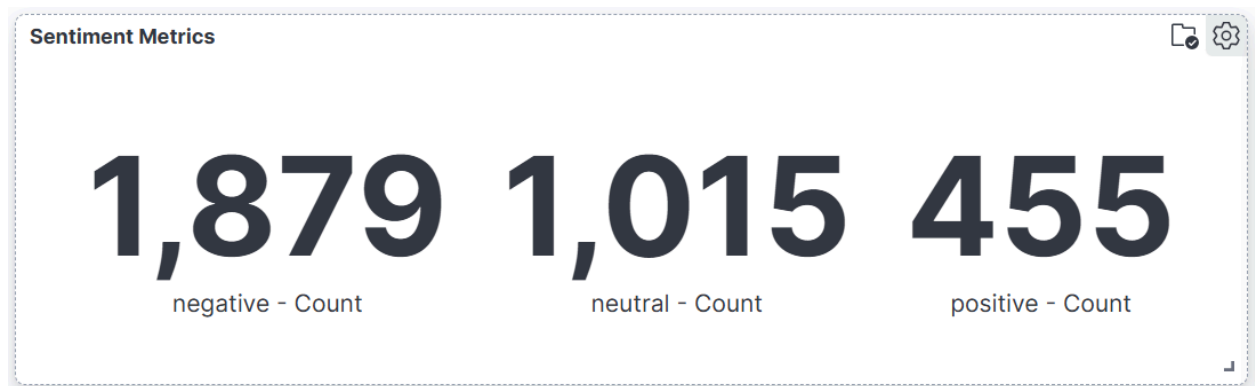


Figure 5.3.2 Sentiment Metrics

- Insight 3: Model confidence is moderate and reflects a balanced three class classifier

As shown in Figure 5.3.3 , the confidence histogram illustrates that the model;s prediction confidence centered around 0.40-0.45, with the bulk of predictions exceeding 0.70. This

distribution is consistent with the expected behaviour of a three-class Logistic Regression classifier, since the model divided probability mass across three classes, a confidence score above 0.33 already indicate a decisive lean toward one sentiment. The histogram shape is right-skewed indicate that while predictions are moderately confident rather than highly certain, they are directionally clear. There are very few of predictions fall below 0.35, meaning the model rarely produced near-random outputs.

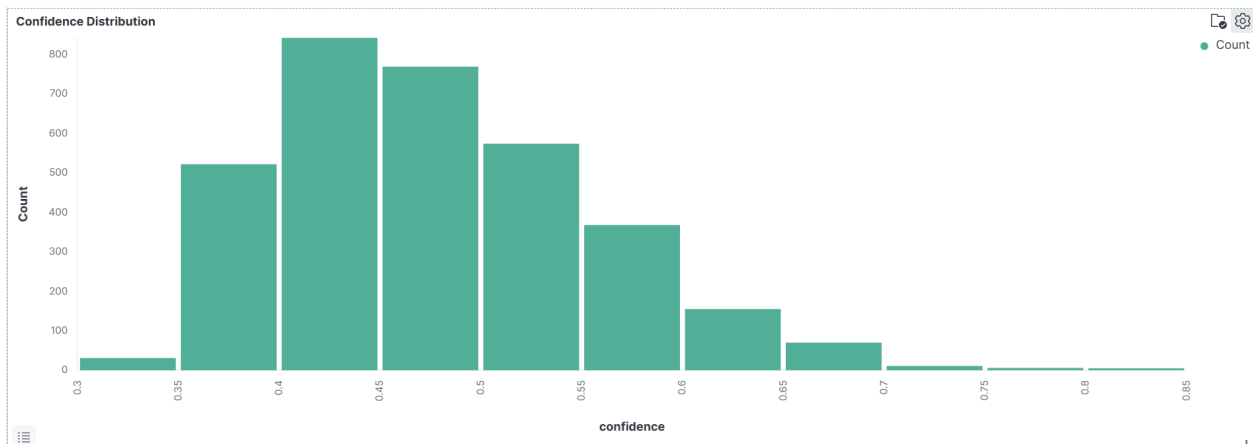


Figure 5.3.3 Confidence Distribution Histogram

- Insight 4: Sentiment data was ingested successfully in real time

As shown in Figure 5.3.4 is a sentiment over time chart where it plots comments volume per sentiment class against the ingestion timestamp. Because the historical BUDI95 dataset was streamed in a single continuous burst, all records share a narrow timestamp window, resulting in three concentrated data points rather than a spread trend line. However, the chart confirms three things: that all three sentiment classes were successfully classified and stored during the streaming run, the relative volumes match the pie chart, and the pipeline correctly timestamp each record as it is processed.

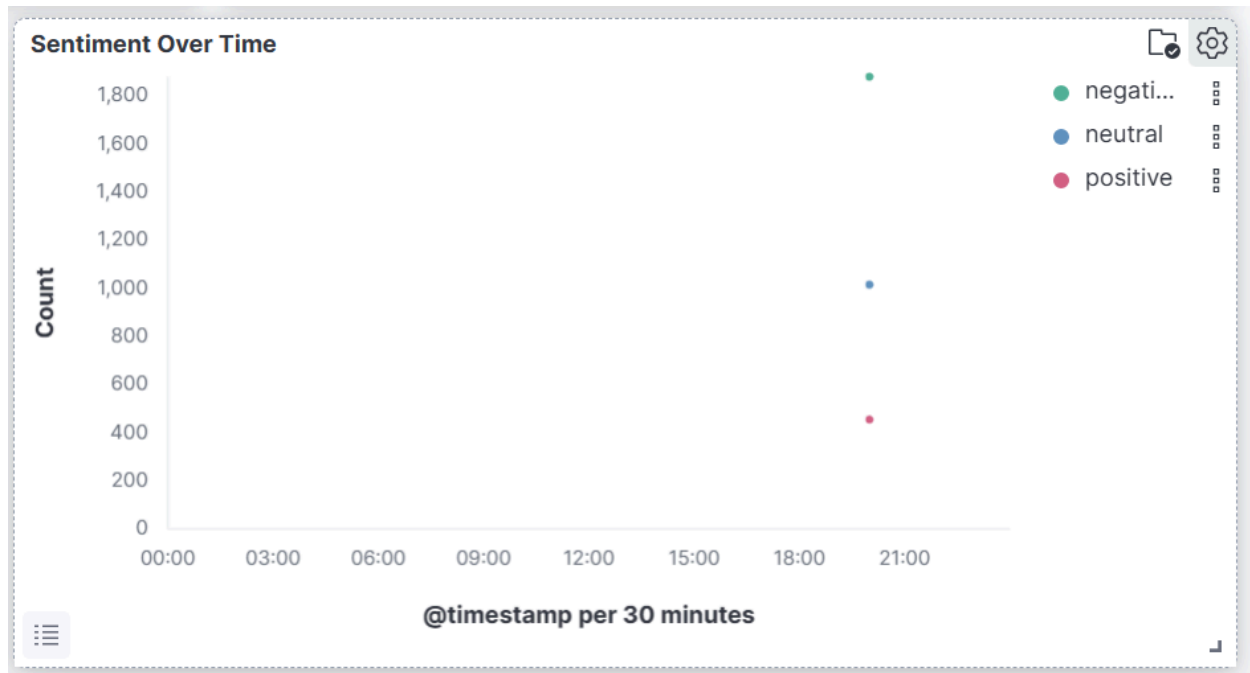


Figure 5.3.4 Sentiment Over Time

- Insight 5: Streaming preserves accuracy while enabling real-time monitoring

Across both batch and streaming modes, the pipeline produced identical classification accuracy of 81.19%. This confirms that the real time architecture introduces no loss of correctness by conducting the same model, applied to the same data, producing the same result regardless of processing mode. The value of streaming lies not in higher accuracy but in its ability to classify and visualize each comment within seconds of arrival, enabling stakeholders to monitor shifts in BUDI95 public sentiment as they happen rather than waiting for batch runs.

- Insight 6: Model deployment reflects a deliberate engineering trade-off

Although XLM-RoBERTa achieved highest classification accuracy during model development, Logistic Regression was selected for the streaming pipeline due to its low inference cost on CPU hardware. This decision reflects a fundamental principle where the best model for accuracy is not always the best model for deployment. The pipeline was designed to support model swapping via a configuration parameter, ensuring that a GPU-backed deployment could run XLM-RoBERTa in the stream without architectural changes.

6.0 Optimization and Comparison

This section documents the optimization techniques used by each model and Spark streaming pipeline, followed by a performance comparison across all models and a contrast between batch and streaming inference modes.

6.1 Model Optimization

Each model required a different optimization approach to address class imbalance in the dataset.

6.1.1 Naive Bayes

Laplace smoothing with $\alpha = 0.1$ was applied to handle zero-probability n-grams. Besides that, no further imbalance was applied, which contributed to poor recall on the minority positive class. The code used to optimize is shown in Figure 6.1.1.

```
Python
nb_model = MultinomialNB(alpha=0.1)
nb_model.fit(X_train, y_train)
y_pred_nb = nb_model.predict(X_test)
```

Figure 6.1.1 Code to Optimize Naive Bayes

6.1.2 Logistic Regression

Logistic Regression was optimized using “class_weight=’balance’” to handle class imbalance by increasing the penalty for minority classes. A regularization parameter of $C=0.5$ was used to reduce overfitting on the small dataset of 3349 samples. In addition, the “saga” solver was used to perform multinomial logistic regression with L2 regularization efficiently. The code used to optimize logistic regression is shown in Figure 6.1.2.

```
Python
lr_model = LogisticRegression(
    C=0.5,                # stronger regularisation for small dataset
    max_iter=1000, solver='saga',
    multi_class='multinomial', class_weight='balanced',
    random_state=SEED, n_jobs=-1)
```

```
lr_model.fit(X_train, y_train)
```

Figure 6.1.2 Code to Optimize Logistic Regression

6.1.3 XLM-RoBERTa

A custom WeightedTrainer was implemented to apply weighted cross-entropy loss to handle class imbalance, with class weights assigned as follows: negative: 0.5418, neutral: 1.2541 and positive: 2.8005. Early stopping if patience = 3 was used to terminate training when validation F1-score no longer improved, with best checkpoint automatically restored. FP16 mixed precision was enabled to improve GPU efficiency during fine-tuning on a T4 GPU. The code used is shown in Figure 6.1.3. Table 6.1 demonstrates the difference between the optimization method for these three models.

Python

```
class WeightedTrainer(Trainer):
    def compute_loss(self, model, inputs, return_outputs=False, **kwargs):
        labels = inputs.pop('labels')
        outputs = model(**inputs)
        loss = torch.nn.CrossEntropyLoss(weight=cw_tensor)(outputs.logits,
labels)

        return (loss, outputs) if return_outputs else loss
training_args = TrainingArguments(
    num_train_epochs = 8,
    learning_rate = 2e-5,
    weight_decay = 0.01,
    warmup_steps = 50,
    metric_for_best_model = 'f1',
    load_best_model_at_end = True,
    fp16 = torch.cuda.is_available(),)
trainer = WeightedTrainer(
    model=xlm_model, args=training_args,
    callbacks=[EarlyStoppingCallback(early_stopping_patience=3)])
```

Figure 6.1.2 Code to Optimize XLM-RoBERTa

Table 6.1 Comparison of Optimization Method Among Three Models

Strategy	Naive Bayes	Logistic Regression	XLM-RoBERTa
Imbalance handling	Class priors	class_weight='balanced'	Weighted cross-entropy
Regularization	Laplace ($\alpha=0.1$)	L2 (C=0.5)	Weight decay (0.01)
Overfitting Prevention	N/A	L2 regularisation	Early stopping (patience=3)
Feature Input	TF-IDF (7,531 features)	TF-IDF (7,531 features)	SentencePiece BPE tokeniser
Training Hardware	CPU	CPU (parallelized)	GPU T4, fp16

6.2 Model Performance Comparison (Machine Learning vs Transformer)

The performance gap between the classical models and XLM-RoBERTa shows a significant difference in text representation. Naive Bays and Logistic Regression both rely on TF-IDF, a bag-of-words representation that ignores word order and treats the tokens as independent features. While this approach works well for clearly expressing sentiment, it struggles with Manglish phrases where the meaning depends heavily on context that TF-IDF cannot capture.

In contrast, XLM-RoBERTa is a transformer-based model pre-trained on 2.5 TB of multilingual text, including Malay. It can encode contextual relationships between words through multiple transformer layers to understand mixed-language phrases without requiring translation. This resulted in an improvement in F1-score and accuracy compared to the other classical models, particularly in minority neutral and positive classes.

The classical models also reflect class imbalance bias. Naive Bayes tend to over-predict the negative class with recall: 0.86 and perform poorly on the positive class with recall: 0.09. logistic regression reduces these issues through class weighting but remains limited by the TF-IDF representation due to dominance of negative. In comparison, XLM-RoBERTa produces a more balanced prediction distribution (negative: 63.0%, neutral: 26.3%, positive: 10.7%), which more closely aligns with the actual dataset distribution.

6.3 Pipeline Optimization

Several design and configuration decisions were made to improve the pipeline performance, reliability, and deployability on local hardware.

1. Batch prediction over row-by-row inference

Rather than classifying one comment per call inside the Spark streaming job, predictions are applied to the entire micro-batch at once using the `predict_batch()` function. The TF-IDF vectorizer transforms all text in a single matrix operation, and the Logistic Regression model classifies the full matrix simultaneously. This reduced per-record overhead significantly where once the model was loaded, the pipeline processed 3,149 records in 0.175 seconds.

2. Lightweight model selection for real-time deployment

XLM-RoBERTa was identified as the best performing model during evaluation but requires GPU acceleration for practical real-time use. Logistic Regression was deployed in the streaming pipeline as an optimization, trading a small accuracy difference for dramatically higher throughput on CPU hardware. The `MODEL_OVERRIDE` environment variable allows the model to be swapped without modifying the pipeline code, maintaining flexibility.

3. Kafka topic decoupling for fault tolerance

Kafka acts as a durable buffer between the producer and Spark. If Spark is temporarily stopped, messages accumulate in the kafka topic and are processed when Spark restarts. Combined with Sparks checkpoint directory, the pipeline can recover from interruptions without data loss or reprocessing.

4. Dockerized infrastructure for reproducibility

All infrastructure components were containerized using Docker Compose as shown in Figure 6.3.2 and Figure 6.3.3, eliminating environment specific installation issues and ensuring the pipeline can be reproduced consistently across machines as shown in Figure 6.3.1. This also enable clean startup and teardown sequences, which is essential for demonstrating the pipeline in a live presentation.

```
query = (parsed.writeStream
        .foreachBatch(process_batch)      Ctrl+u TRAE Add to chat
        .option("checkpointLocation", "./_chk_sentiment")
        .start())
```

Figure 6.3.1 Code snippet from spark_streaming.py

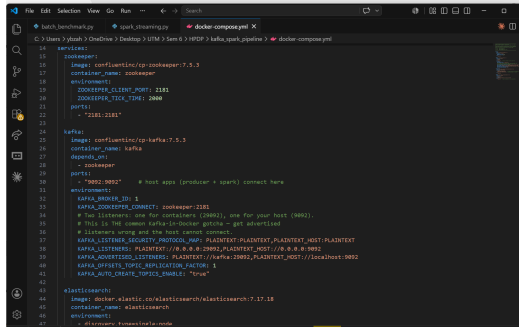


Figure 6.3.2 Container configuration

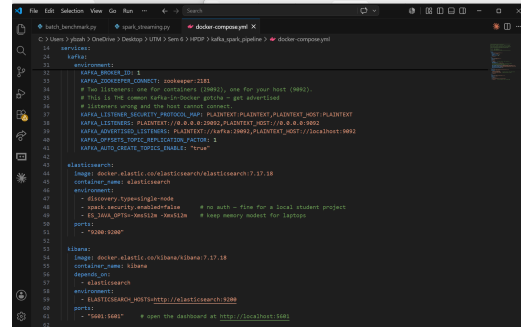


Figure 6.3.3 Container configuration

6.4 Batch vs Streaming Performance Comparison

The same 3,349 BUDI95 YouTube comments were processed in both batch and streaming mode. Results are presented as below in Table 6.4 :

Table 6.4 Comparison table Batch vs Streaming

Metric	Batch Mode	Streaming Mode
Records processed	3,349	3,349
Total processing time	0.065s	3.973s
Throughput	51,509 rec/s	~843 rec/s
Accuracy	81.9%	81.19%
CPU usage	86.9% (single core)	~10.8% (across stack)
Memory usage	144.0 MB (single process)	~2.8 GiB (across stack)

a. Total processing time

Batch mode processed all 3,349 records in 0.065 seconds, while streaming required 3.973 seconds end-to-end. The streaming overhead is dominated by the one-time model loading cost in the first micro-batch (3.798 seconds), which accounts for 95% of the total streaming time. Excluding this warm up, the remaining records were processed in just 0.175 seconds, meaning that when the model is in steady-state, streaming is highly efficient once the model is resident in memory.

b. Throughput

Batch mode achieved 51,506 records per second through direct in-memory vectorization with no inter-system communication overhead, meanwhile streaming throughput was approximately 843 records per second, reflecting the combined overhead of Kafka message serialization, network transfer between producer and Spark, and micro-batch scheduling. Steady-state streaming throughput reached approximately 17,999 records per second, indicating that the pipeline is capable of high speed classification once initialization is complete.

c. Classification accuracy and consistency

Batch mode produced identical classification accuracy of 81.19% confirming that the streaming pipeline introduces no loss of correctness. Since the same Logistic Regression model and TF-IDF vectorizer were applied in both modes, therefore predictions are fully consistent regardless of processing architecture.

d. Resource usage

In batch mode, the python process consumed 86.9% of a single CPU core and 144.0MB of memory, but only for 0.065 seconds before terminating which the footprint is intense but extremely brief, and no infrastructure remains running afterward. Meanwhile, in streaming mode, the four container stack consumed a combined ~10.8% CPU and ~2.9 GiB of memory as measured via docker stats during active streaming. Streaming has lower peak intensity than batch, but sustains continuously as long as the pipeline is running.

7.0 Conclusion and Future Work

This section presents the conclusion and future work of the project.

7.1 Conclusion

This project successfully collected 3,448 YouTube comments from 16 videos related to Malaysia's BUDI95 fuel subsidy policy. After preprocessing, 3,349 high-quality comments remained for analysis. The XLM-RoBERTa model was then used to assign sentiment labels to each comment. This dataset contained a balanced mix of English, Malay, and Manglish comments. This shows the multilingual nature of online discussions in Malaysia. The labelled dataset showed that negative sentiment accounted for 61.5% of all comments, which makes it the dominant sentiment.

Three sentiment classification models were trained and evaluated, including Multinomial Naive Bayes, Logistic Regression and XLM-RoBERTa. XLM-RoBERTa was selected as the best model with a weighted F1-score of 0.7381 and accuracy of 74.03% due to its large-scale pretraining on over 100 100 languages, including Malay which allow it to handle multilingual and Manglish comments without translation.

Furthermore, this project has successfully implemented a real time sentiment analysis pipeline that processed the 3,349 classified BUDI95 comments through Apache Kafka, Apache Spark Structured Streaming, Elasticsearch, and Kibana. The pipeline was deployed on local hardware using Docker Compose, ensuring reproducibility across environments. The pipeline deployed Logistic Regression for real-time inference while XLM-RoBERTa achieved superior accuracy with a weighted F1-score of 0.7381, its transformer based inference is computationally intensive on CPU hardware and unsuitable for low latency streaming. Lastly, the Kibana dashboard shows that negative sentiment dominated using consistent labelled dataset distribution.

Overall, this project demonstrated a complete, scalable, and accurate real-time sentiment analysis system for Malaysian social media content can be built using open-source technologies, delivering actionable insight on public opinion toward BUDI95 through interactive visualization to non-technical stakeholder.

7.2 Future work

In terms of data collection, future work could expand the dataset by scraping comments from other platforms such as X, Facebook, or Thread, where public discussions about Malaysian fuel policies are also active. This would provide a more complete picture of public opinion beyond YouTube. The dataset could also be updated continuously over time to track how sentiment shifts as new policy announcements are made.

For preprocessing, future improvements could include a more sophisticated Malay slang dictionary that covers newer internet slang and a better language detection approach specifically trained on Manglish text. Adding emoji sentiment mapping could also help retain the emotional signal carried by emojis, which are commonly used in social media comments but are currently removed during cleaning.

While XLM-RoBERTa achieved the best performance in this project, future iterations could explore alternative multilingual models such as mBERT fine-tuned, specifically on larger Malaysian social media corpus or domain-adapted models trained specifically for BUDI95-related vocabulary. Furthermore, experimenting with large variants, such as XLM-RoBERTa-Large or newer multilingual transformer architectures may further improve the model performance because they could be particularly useful to improve the classification of the minority positive class, which remained the most challenging across all evaluated models.

Applying a real-time live data ingestion where integrating the YouTube Data API v3 as a live Kafka producer would enable genuine continuous monitoring because new BUDI95 comments would flow into the pipeline as they are posted. By doing this, stakeholders could view the sentiment in real-time as policy announcements or news events unfold.

Furthermore, accelerating GPU streaming with XLM-RoBERTa. Deploying the pipeline on GPU enabled cloud infrastructure such as AWS EMR, Google Cloud Dataproc, or Databricks would allow XLM-RoBERTa to be used directly in the streaming path, combining its multilingual accuracy with real-time throughput.

8.0 References

- Barbieri, F., Camacho-Collados, J., Espinosa-Anke, L., & Neves, L. (2022). XLM-T: Multilingual language models in Twitter for sentiment analysis and beyond. In *Proceedings of the 13th Language Resources and Evaluation Conference* (pp. 258–266). European Language Resources Association.
<https://huggingface.co/cardiffnlp/twitter-xlm-roberta-base-sentiment>
- GeeksforGeeks. (2025, July 23). *Best data cleaning techniques for preparing your data*. GeeksforGeeks.
<https://www.geeksforgeeks.org/data-science/best-data-cleaning-techniques-for-preparing-your-data/>

9.0 Appendices

Github Repository Link:

<https://github.com/drshahizan/HPDP/tree/main/2526/project/p2/Fast%26Furious>